COS30018 - Intelligent System

Project: Stock Price Prediction System

Task C.4 - Machine Learning - Part 1

Student Name: Ngo Sy Vuong
Student ID: 105551480
Tutor: Nguyen Manh Toan
Tutorial Session: Wednesday 8:00 - 12:00
Semester: May - July 2026

SWINBURNE UNIVERSITY OF TECHNOLOGY

Table of Contents

Environment Setup

To successfully replicate and run the stock price prediction model, please fulfill the following structural and software dependencies.

1. Environment

The system is built on an isolated virtual environment (venv) to ensure dependency abstraction and eliminate version conflicts with global system libraries:

- Core Runtime: **Python 3.11.x (64-bit)**
- Execution Interface: **Windows Command Prompt (CMD)**

2. Dependencies

tensorflow	Core Deep Learning framework used to compile structure and execute the LSTM network
yfinance	Automated financial data retrieval tool used to fetch historical structural data for CBA.AX via the Yahoo Finance API
scikit-learn	Provides data preprocessing mechanics
pandas	Sequential series concatenation, index cleaning and CSV file extraction
numpy	Low-level vectorized array manipulation, matrix dimension reshaping and numeric tensor optimization
matplotlib	Render visual evaluations
mplfinance	Specialized financial visualization extension built on Matplotlib
seaborn	Statistical data visualization framework built on top of Matplotlib

3. Deployment

To reconstruct this environment layout from scratch, run the following sequential commands within the target file directory: **(make sure you have installed Python 3.11)**

```
# Initialize the Python 3.11 Isolation layer
py -3.11 -m venv venv

# Activate the localized scripts block (Windows)
venv\Scripts\activate.bat

# Synchronize deployment prerequisites
pip install -r requirements.txt
```

Executive Summary

The purpose of Task C.4 is to automate the construction stage of the deep learning network by creating a function called model_training_setup.py. Instead of manually typing out layers, neurons and dropouts each time we want to experiment, this function assembles a model based on chosen parameters.

Specifically, Task C.4 delivers a function that accepts a target recurrent cell type (can be LSTM, GRU or RNN), an array of hidden layer sizes (like [50, 50, 50]) and the data input shape (like 60 days x 5 features). It then automatically handles the technical configurations: Mapping string inputs to Keras objects, setting up sequence continuity (return_sequences), injecting dropout layers to prevent overfitting, attaching a final prediction layer and compiling the network with the Adam optimizer and the MSE (Mean Squared Error) loss.

Ultimately, Task C.4 does not train or evaluate the model, it just automates the construction pipeline, combining into a single function in a separate file so that the main program (stock_prediction.py) will just need to call that function.

Stock Prediction Version 4

1 - Initialize Model container and calculate layer depth

The function first creates an empty Keras Sequential object, Sequential here means the network layers will sit in a straight, linear stack, the data will go in the bottom, move up layer by layer and spit a guess out the top.

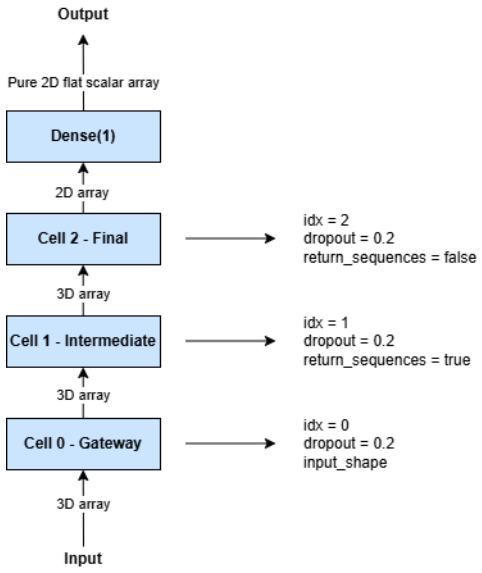

Figure 1 - Dimensional Transformation Map

The second line runs the built-in len() method on layer_sizes. For example, if the list passed is [50, 50, 50], the variable num_layers is evaluated to 3.

```
# layer_type: A string text (LSTM / GRU / RNN)
# layer_sizes: A list of numbers like [50, 50, 50]
# input_shape: The size of the incoming data grid (60 days x 5 features)
# dropout_rate: Default setting
# learning_rate: Default setting
def construct_dynamic_dl_model(layer_type, layer_sizes, input_shape, dropout_rate=0.2, learning_rate=0.001):

    # Initialize an empty Keras model container
    # Sequential: The network layers will sit in a straight, linear stack
    # (data goes in the bottom, moves up layer by layer and spits a guess out the top)
    model = Sequential()

    # Counts the items in the size list
    # For example, [50, 50, 50] will be num_layers = 3
    num_layers = len(layer_sizes)
```

Figure 2 - Initialize model container and calculate layer depth

2 - Check the order and pick the machine cell

A lookup catalog maps human readable string keys directly to uninstantiated layer class handles. By passing the string variable layer_type (can be LSTM, GRU or RNN), the function extracts the explicit structural object class reference from the map and assigns it to cell_object.

```
layer_mapping = {
    'LSTM': LSTM,
    'GRU': GRU,
    'RNN': SimpleRNN
}
cell_object = layer_mapping[layer_type] # For example: select LSTM then cell-object = LSTM
```

Figure 3 - Check the order and pick the machine cell

3 - Stacks the layers and dropouts

The function stacks the layers and dropouts using a ‘for’ loop. This loop leverages enumerate() to traverse the configuration list, extracting both the loop sequence index (idx) and the node width (units) for each pass. For each hidden layer, a layer_kwargs is instantiated. In Python, kwargs stands for keyword arguments, and a layer_kwargs acts as an empty packing pouch. Inside the loop, kwargs is treated as a standard Python dictionary, where the parameter name becomes the key and the passed value becomes the value.

```
# idx: The layer number index (Layer 0, then Layer 1, then Layer 2)
# units: Number of neural cells belong in that specific layer (like 50)
for idx, units in enumerate(layer_sizes):

    # Used to store customization settings for the current layer being built
    # In C.4, it just notes how many hidden units to use
    layer_kwargs = {'units': units}

    # This handles the entry gate rule
    # Only the absolute first layer needs to know the shape of the raw data table
    # Next layers do not get this setting because they automatically adjust
    # to whatever the layer below them outputs
    if idx == 0:
        layer_kwargs['input_shape'] = input_shape

    # When stacking multiple neural layers, they have to communicate differently
    # depending on where they sit in the stack
    # Intermediate Layers (return_sequences = True):
    #   If there are more recurrent layers waiting above the current one,
    #   this layer cannot just pass a single summary guess, it must hand over
    #   its full 3D timeline history so that the next layer up can analyze it
    # The Final Layer (return_sequences = False):
    #   When the loop reaches the absolute top recurrent layer, it does not have
    #   another recurrent layer following it, so it drops the 3D timeline history
    #   and compresses all its findings into a 2D vector
    if idx < num_layers - 1:
        layer_kwargs['return_sequences'] = True
    else:
        layer_kwargs['return_sequences'] = False

    # Burst open the layer_kwargs and feed its contents straight into the layer
    model.add(cell_object(**layer_kwargs))

    # Set the dropout_rate for the layer
    # As mentioned in C.1 report, if dropout_rate = 0.2, the system will randomly
    # shut off 20% of the layer's neural brains on every single step.
    # This stops the model from memorizing the exact training charts and forces
    # it to learn flexible, general market trends instead
    model.add(Dropout(dropout_rate))
```

Figure 4 - The loop used to stack the;layers and dropouts

The Input Gate Rule will be applied to the first layer of the stack: The recurrent nodes require explicit baseline coordinate metrics during their entry pass. The condition (if idx == 0) ensures that the input_shape matrix bounds (60 days x 5 features) are strictly coupled to the gateway layer. Subsequent layers skip this block, automatically parsing their incoming shapes from the internal dimensions calculated by the lower layer outputs.

Recurrent neural layers process sequential data across daily time steps. When stacking multiple layers, intermediate layers must pass their entire chronological timeline forward so that the next layer up can continue analyzing temporal pattern parts. The function forces return_sequences=True for all intermediate layers. When the loop arrives at the final recurrent layer, there are no more recurrent layers left to read time steps. The function makes return_sequences=False, stripping away the time-dimension index and compressing the pattern into a flat vector summary (from 3D to 2D).

The statement cell_object(**layer_kwargs)) unpacks the settings dictionary directly into the active recurrent class constructor. Once built, the layer is appended to the Sequential stack. Immediately following this, a Dropout layer is injected. As established in Task C.1, a dropout_rate=0.2 forces the network to randomly deactivate 20% of its active nodes during each training pass. This is used to prevent specific weights from memorizing the exact historical chart (overfitting. Forcing the system to isolate flexible, generalized market trends.

4 - Add the final top layer and compile

After exiting the loop, the model holds a flat 2D output shape. The function welds a standard, fully-connected Dense layer with exactly 1 node right onto the peak of the stack. This acts as a final linear aggregator, it reads the multi dimensional feature vector sent by the final hidden layer, reads the weights and biases, and compresses the value down into a single decimal number representing the next day's predicted stock price.

```
# After having done compliling the layers, it puts a final node right onto the top
# This node reads the flat pattern summary vector sent by the last recurrent layer
# and calculates a value: The next day's final predicted stock price
model.add(Dense(units=1))

# Apply Adam optimizer to the model before training
# More detail about the Adam optimizer will be included in the C.4 Report
optimizer = tf.keras.optimizers.Adam(learning_rate=learning_rate)
model.compile(optimizer=optimizer, loss='mean_squared_error')

return model
```

Figure 5 - Adding the final top layer and compile after having done the loop

The final phase wires the system for training. It builds the Adam optimizer, configuring it with a specified learning_rate step modifier to guide gradient descent updates. The .compile() function links the optimizer to the layer weights and assigns the MSE (Mean Squared Error) equation to measure how far the model guesses drift from the real stock prices. The fully constructed and compiled model instance is then returned to the main workspace (stock_prediction.py), ready for the .fit() execution, which is the training phase. For more information about the Adam optimizer, please see the Appendix section.

Experiment with different models

1 - Experiment 1: Cell Type Comparison (LSTM - GRU - RNN)

The table below displays the output of each model with the same setting: layer_sizes=[50, 50, 50], epochs=25, batch_size=32 and learning_rate=0.001.

Model	MSE (Training Loss)	Average Execution Speed per Epoch	Predicted Next-Day 'Close' Valuation
LSTM	0.0048	~28 ms/step	$106.84
GRU	0.0042	~32 ms/step	$115.18
RNN	0.0170	~11 ms/step	$110.30

The first run is for LSTM, this model achieved a good MSE while maintaining a balanced processing velocity of ~28 ms/step. The LSTM chart shows excellent trend direction tracking, the green prediction curve tightly mirrors the true macro trajectory of CBA.AX , cleanly mimicking major historical shifts with minimal directional lag. (See Figure 6)

LSTM has an internal cell state regulated by 3 unique mathematical lock gates: an input gate, a forget gate and an output gate. These mechanisms allow the network to preserve gradients across all 60 lookback days, allowing it to remember structural long-term asset trends without losing training stability.

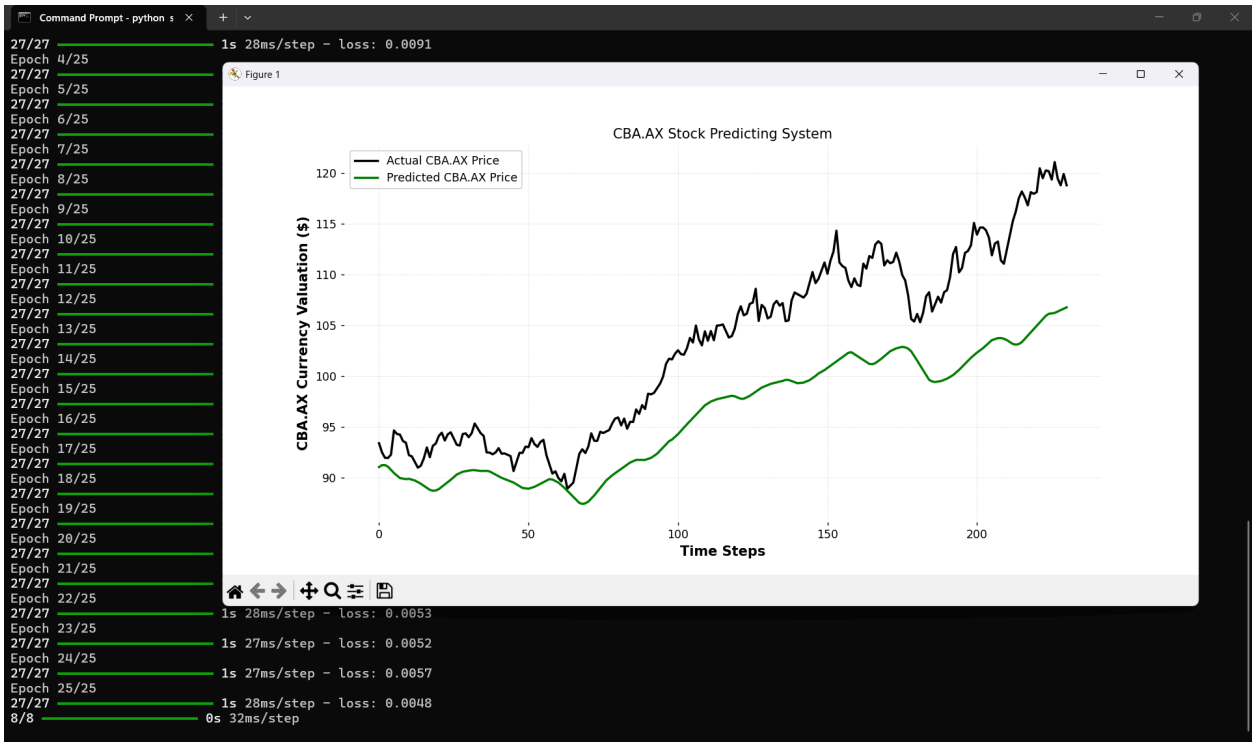

Figure 6 - Experiment 1 output of LSTM

The second run is for GRU, this model achieved the best MSE among the 3 models while maintaining a balanced processing velocity of ~32 ms/step. The GRU chart displays a highly adaptive, responsive prediction line, it tracks sudden trend changes with greater sensitivity than the LSTM, showing that the model successfully minimized its loss bounds without exhibiting signs of structural overfitting. (See Figure 7)

GRU architectures optimize the standard LSTM framework by merging the cell state and hidden state together. It replaces the traditional 3 gates with just 2: an update gate and a reset gate. Because it has fewer internal parameters, it often converges onto a lower error floor faster and more cleanly than an LSTM when handling noisy, medium sized financial tabular arrays.

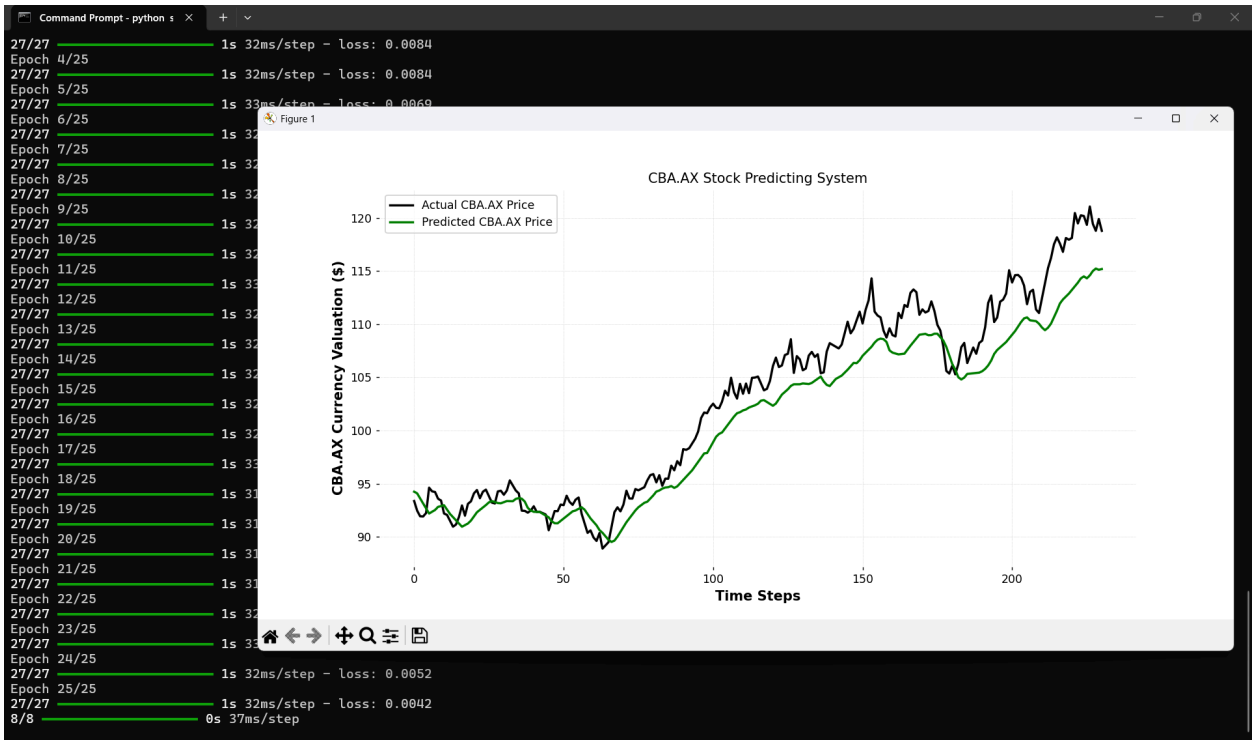

Figure 7 - Experiment 1 output of GRU

The RNN (SimpleRNN) model completed training with the highest final MSE (0.0170), which is roughly 4 times worse than both the LSTM and GRU models. However, it has the fastest processing throughput with just ~11 ms/step. Looking at the RNN chart, the green line fails to track sudden, high frequency price turnarounds, especially between time steps 150 and 200. Instead, it generates a delayed, smoothed out approximation because it can no longer recall the long-term historical variance from earlier time steps. (See Figure 8)

A standard RNN updates weights using a simple, un-gated recurrent feedback loop. Over a long lookback sequence window of 90 days, computing gradients across continuous matrix multiplications triggers exponential decay, leading to severe vanishing gradients.

In addition, the vanishing gradient problem is a major obstacle that occurs when training deep neural networks and standard RNNs. It happens during backpropagation, the phase where a network calculates errors and sends them backward to update its weights. In a deep network, as the error signal (gradient) travels backward from the output layer to the earliest layers, it shrinks exponentially. By the time it reaches the first few layers, the gradient has become close to zero, causing the weights in those initial layers to stop updating. Consequently, the earliest parts of the network stop learning.

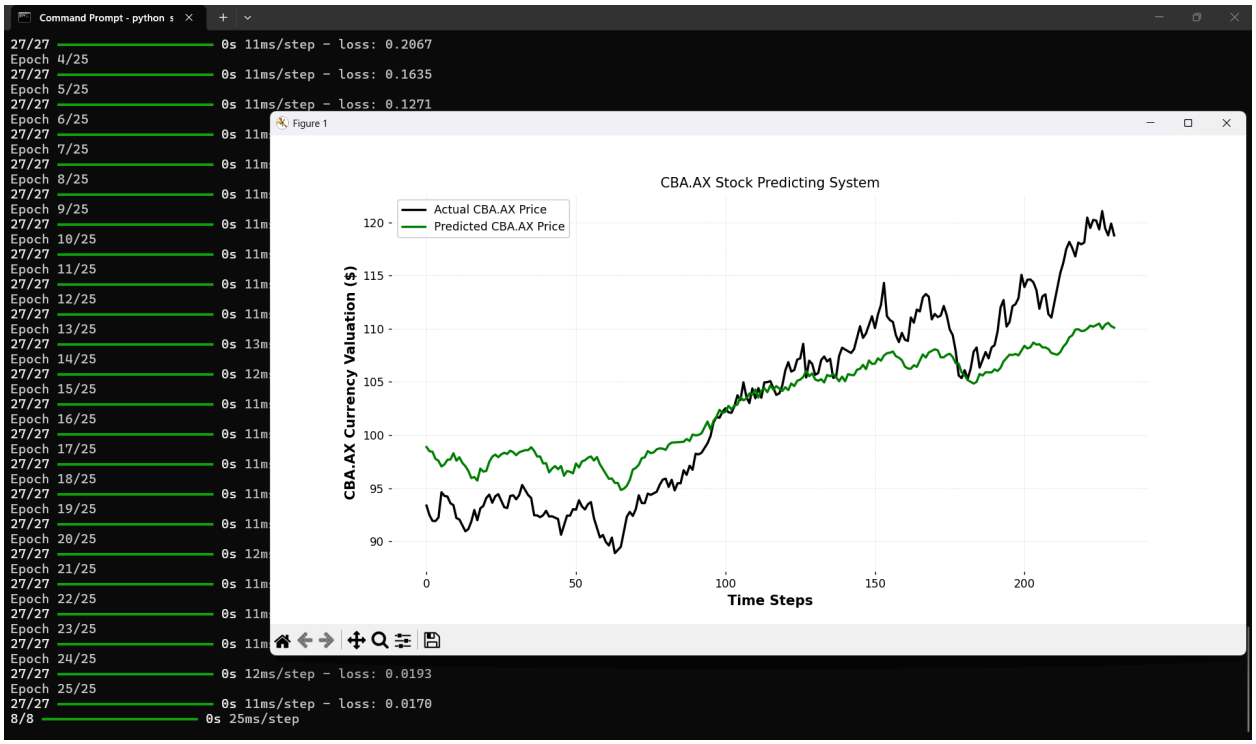

Figure 8 - Experiment 1 output of RNN

2 - Experiment 2: Structural Capacity and Network Depth (layer_sizes)

The table below displays the output of each layer_sizes with the same setting: layer_type='LSTM', epochs=25, batch_size=32 and learning_rate=0.001.

Model	MSE (Training Loss)	Average Execution Speed per Epoch	Predicted Next-Day 'Close' Valuation
layer_sizes=[50]	0.0036	~9 ms/step	$116.26
layer_sizes=[50, 50]	0.0052	~19 ms/step	$112.22
layer_sizes=[128, 64, 32]	0.0052	~35 ms/step	$106.23

Run 1 minimized training error to the lowest level among all 3 configurations (0.0036) with a fast execution footprint of ~9 ms/step. The green line shows a highly reactive tracking. It mirrors minor, high-frequency price shifts accurately. This high sensitivity to late stage sample spikes introduces a risk of local overfitting, where the model responds heavily to immediate terminal momentum at the expense of macro trend stability.

In a single layer recurrent network, backpropagation calculates gradients across a direct path without navigating intermediate layer transitions, enabling rapid convergence. However, a shallow topology has limited representational capacity. It captures immediately preceding local asset movements well but lacks multi tiered abstract features needed to separate long-term structural trends from short-term market noise.

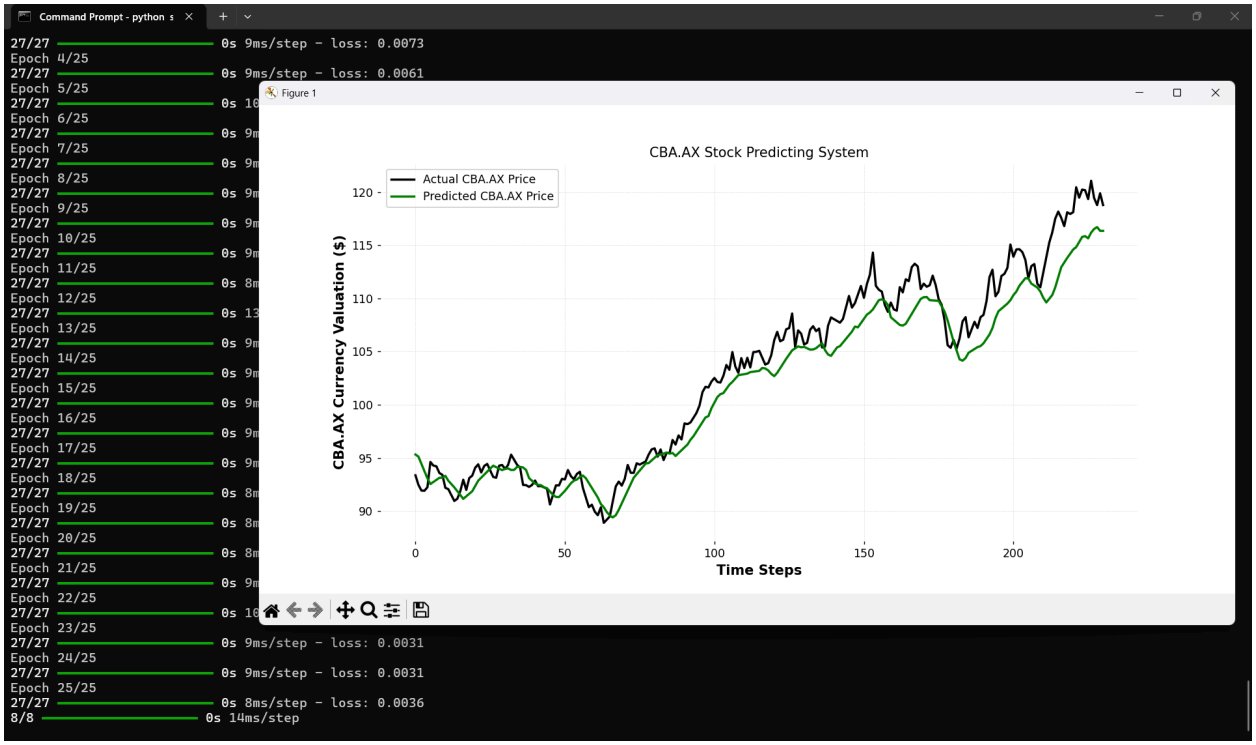

Figure 8 - Experiment 2 output of Run 1 with layer_sizes=[50]

Run 2 converged at a stable loss of 0.0052, while processing steps required ~19 ms due to the expanded backward pass through the stacked parameters. The green line in the graph follows the trend smoothly. This projection sits safely between the cases run in run1 and run 3.

Introducing a second recurrent layer establishes a structural abstraction hierarchy. The primary layer processes raw 60 days sequence features while the secondary layer transforms those representations into higher level temporal patterns. This layered structure behaves like an implicit regularizer, filtering out high frequency market noise and preventing the model from reacting too aggressively to sudden price spikes.

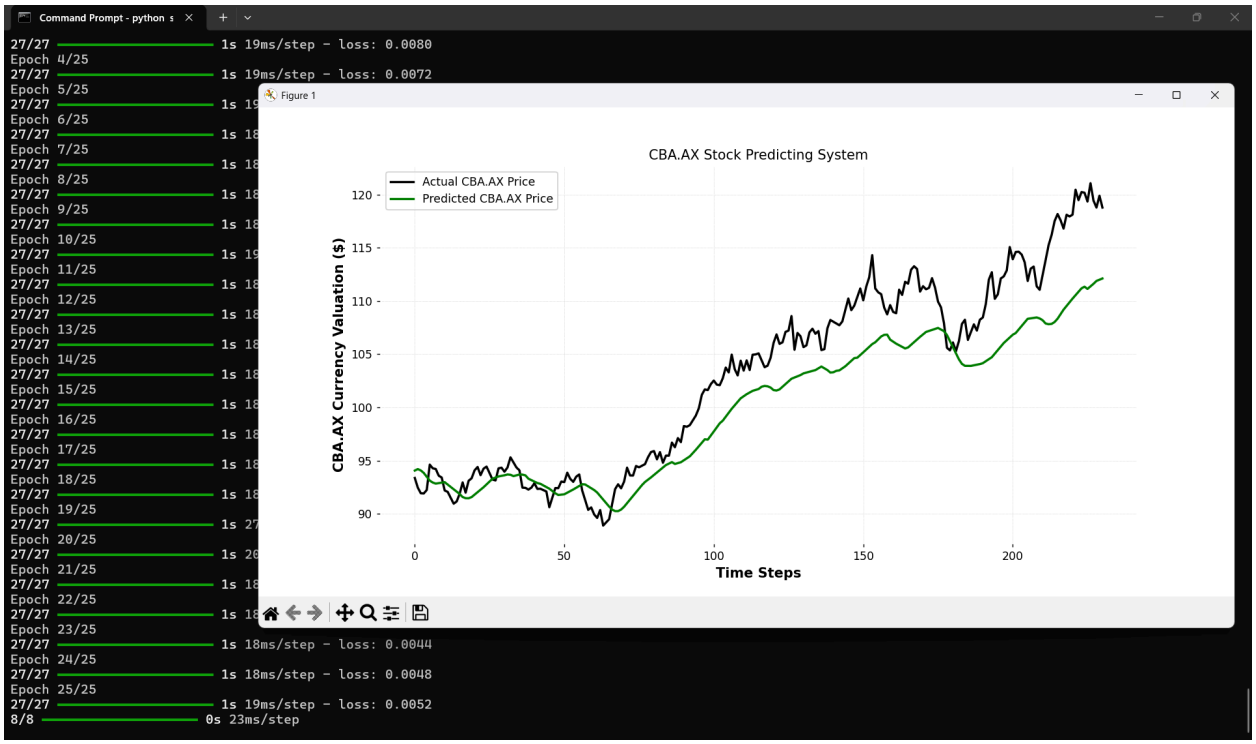

Figure 9 - Experiment 2 output of Run 2 with layer_sizes=[50, 50]

Run 3 matched the training loss of Run 2 (0.0052) but required a heavier processing footprint of ~35 ms/step due to the increased parameter volume. The prediction curve displays substantial directional smoothing and a distinct downward lag relative to the actual price chart. By the end of the timeline, as the true equity price surges upward, the deep model remains heavily anchored to historical averages.

While a tapered structure (128 - 64 - 32) increases the network's overall representational capacity, it introduces a significant structural penalty to the gradient descent path. Propagating errors back through 3 separate LSTM layers can cause recurrent information to decay or become overly compressed by the time it reaches the gateway layer. This results in structural underfitting, where the network stabilizes prematurely at a higher error floor because its deepest parameters stop receiving meaningful update vectors.

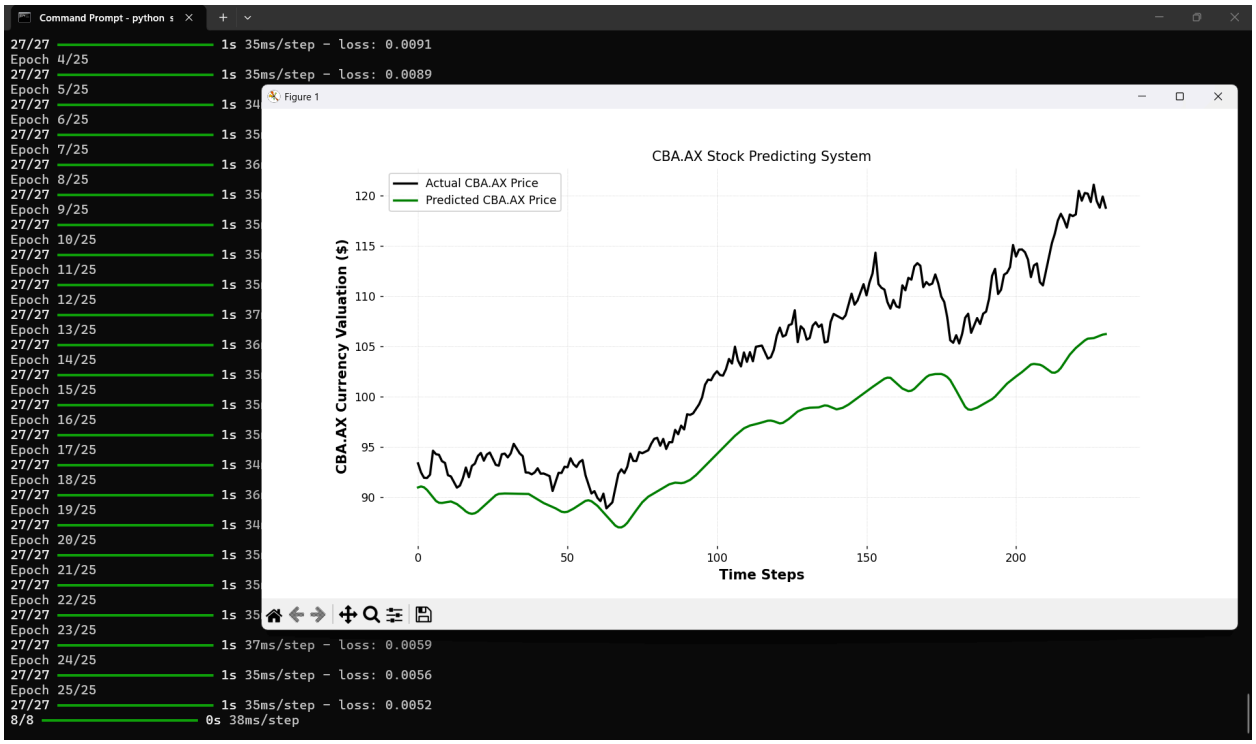

Figure 10 - Experiment 2 output of Run 3 with layer_sizes=[128, 64, 32]

3 - Experiment 3: Optimization Pacing Strategy (batch_size)

The table below displays the output of each layer_sizes with the same setting: layer_type='LSTM', epochs=25, layer_sizes=[50, 50] and learning_rate=0.001.

Model	MSE (Training Loss)	Average Execution Speed per Epoch	Predicted Next-Day 'Close' Valuation
batch_size=16	0.0035	~29 ms/step	$116.26
batch_size=32	0.0045	~29 ms/step	$112.22
batch_size=128	0.0065	~68 ms/step	$106.23

Configuring the matrix chunking to a restrictive window of 16 samples yielded the lowest final training error within this group (0.0035), executing with a localized step velocity of ~29 ms/step. A smaller batch allocation dictates that the Adam optimization engine calculates gradient vectors and updates network weights at a significantly higher frequency per epoch. This operational density introduces stochastic gradient noise.

The green prediction curve showcases a highly reactive tracking capability, matching granular variations of the real asset line and responding accurately to the terminal upward pricing trend,

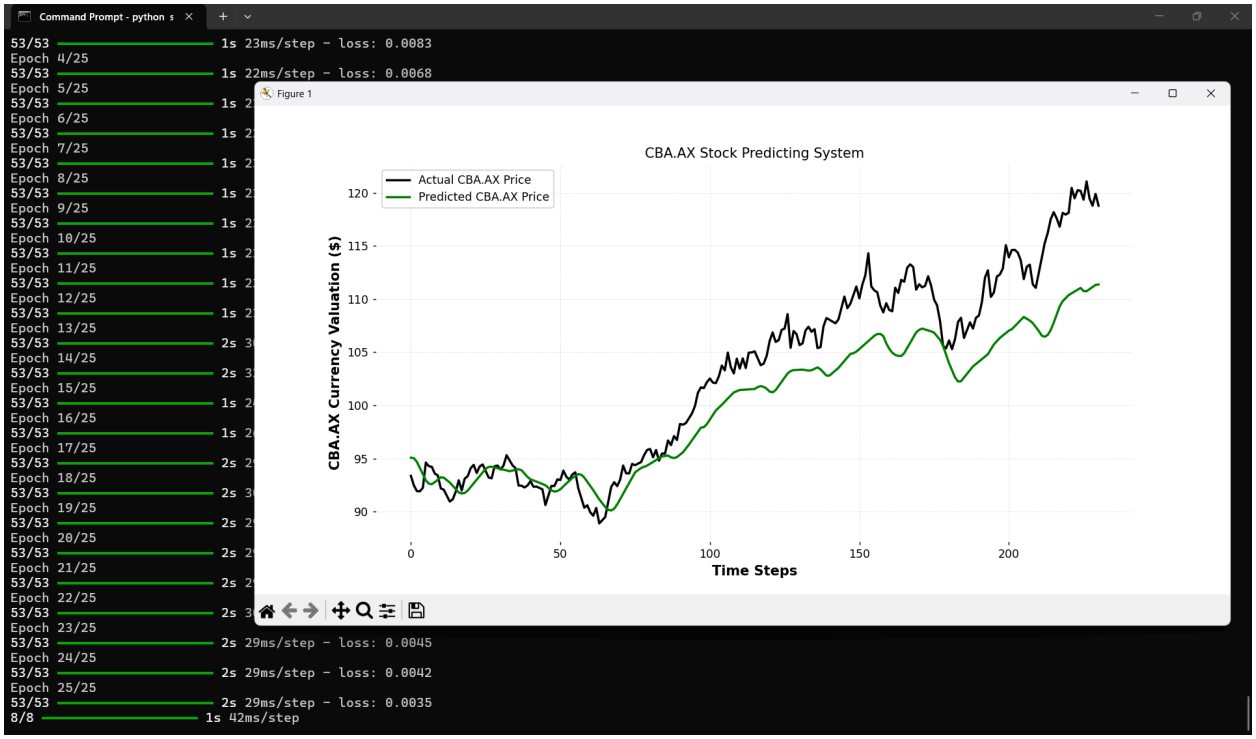

Figure 11 - Experiment 3 output of Run 1 with batch_size=16

Run 2 converged on a stable medium loss value of 0.0045, maintaining a matching hardware step speed of~28ms/step. A batch size of 32 represents the classic industry baseline for training deep recurrent architectures, providing a robust compromise between stochastic exploration and gradient smoothness.

The update vectors calculated per step are sufficiently clean to prevent extreme, erratic parameter jumps, yet they retain enough stochastic energy to keep the weights moving steadily down the loss surface. The visualization demonstrates excellent trend smoothing across the middle section of the time series. While it filters out minor daily market fluctuations, it remains flexible enough to track the macro-trajectory of CBA.AX reliably.

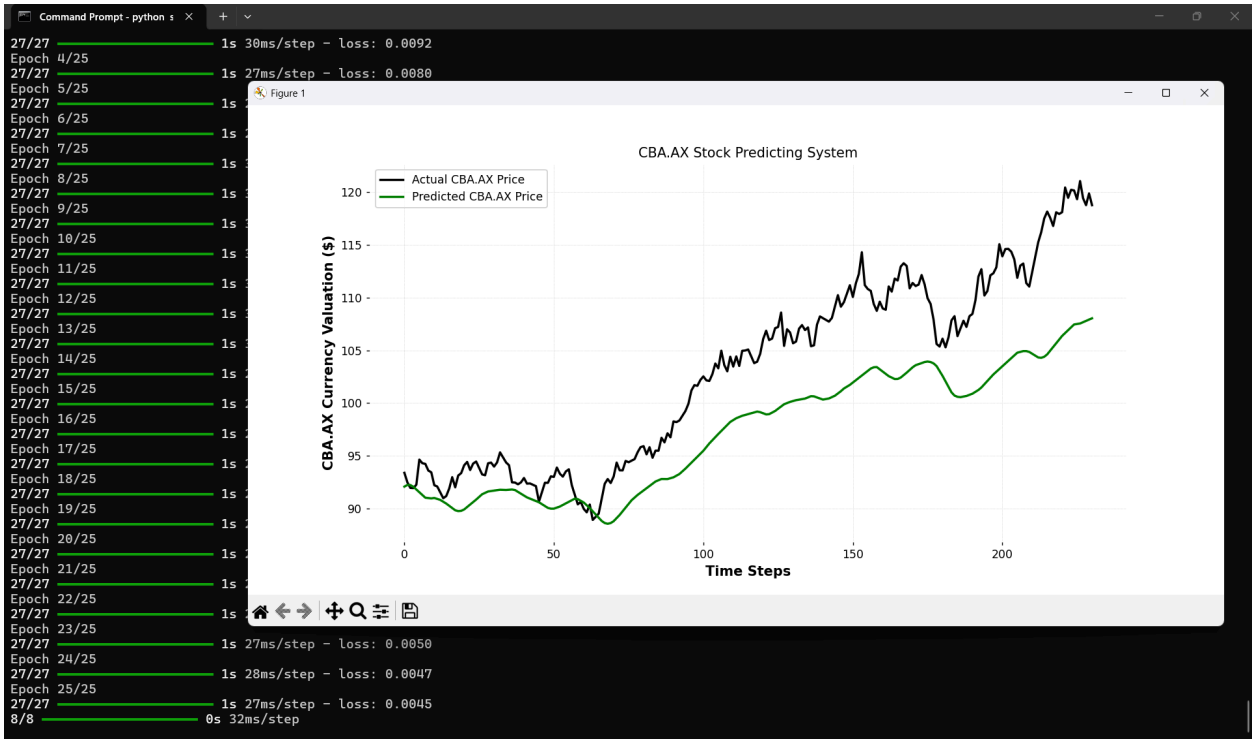

Figure 12 - Experiment 3 output of Run 2 with batch_size=32

Run 3 recorded the poorest optimization performance, stabilizing at a high loss floor (0.0065) and displaying an increased step time profile of ~68 ms/step. When the batch size is expanded to 128, the network processes massive chunks of the data array simultaneously before performing a weight adjustment. This heavily averages out the gradient steps, eliminating the useful stochastic noise found in smaller batches.

When you use a very large batch size, the model averages out too much data at once. Because everything is averaged, the sharp daily ups and downs of the stock market vanish. The mathematical directions given to the model (the gradients) become completely flat, uniform, and boring. Because these directions are so flat and uniform, the model's training steps become highly cautious. A neural network's error map is full of tiny potholes and fake valleys (local ruts). Without any sudden, sharp changes in direction to bounce it out, the model falls into the very first small pothole it hits and freezes there. It lacks the energetic, sharp changes needed to break free and find the true, deepest valley. This frozen state is what is called gradient stasis. Because the model's internal weights got stuck early on and stopped learning, it became under-responsive (lazy).

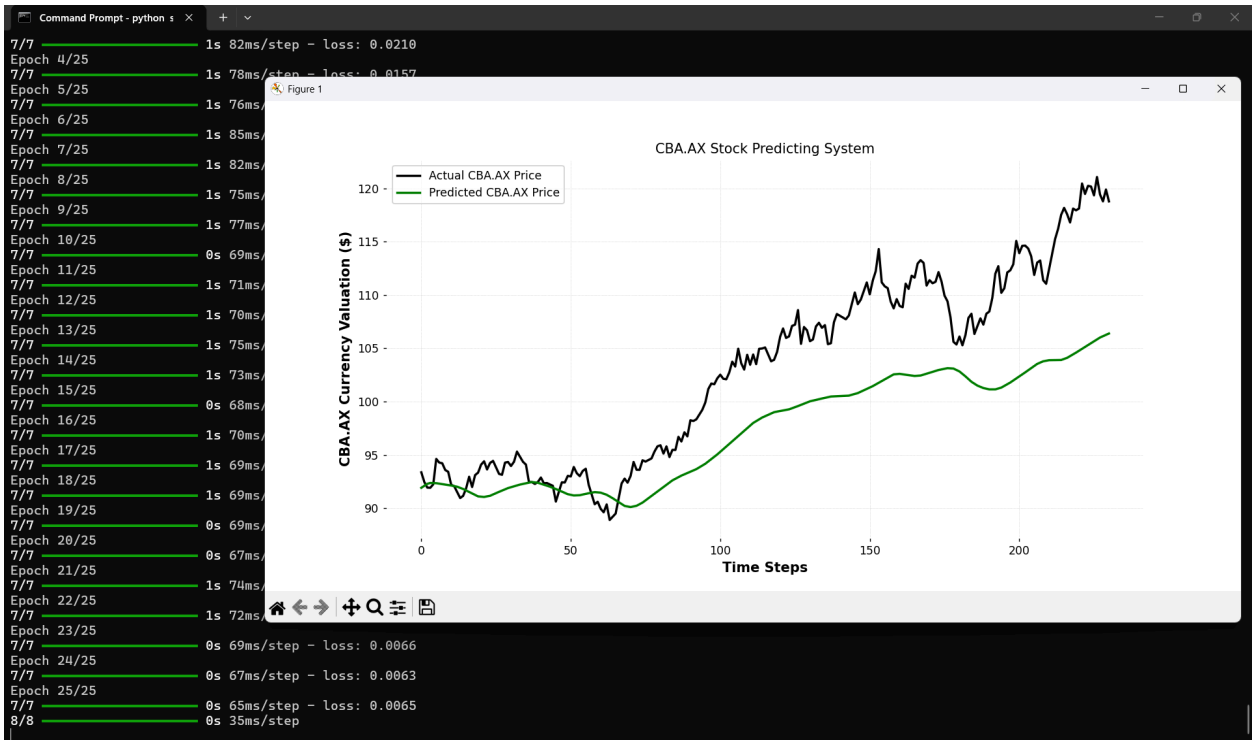

Figure 13 - Experiment 3 output of Run 3 with batch_size=128

4 - Experiment 4: Convergence Velocity Verification (epochs)

The table below displays the output of each layer_sizes with the same setting: layer_type='LSTM', batch_size=32, layer_sizes=[50, 50] and learning_rate=0.001.

Model	MSE (Training Loss)	Average Execution Speed per Epoch	Predicted Next-Day 'Close' Valuation
epochs=10	0.0060	~34 ms/step	$106.07
epochs=25	0.0045	~27 ms/step	$106.10
epochs=100	0.0021	~27 ms/step	$109.56

For run 1, the green line is noticeably smooth and heavily muted. It fails to match the dynamic daily fluctuations of the real asset price. Because the hidden layer weights lack the tuning required to track momentum, the model lags far behind the late stage upward surge.

Interrupted execution at 10 epochs prevents the Adam optimization engine from successfully navigating down the error surface. Early in the training lifecycle, the network weights are still adjusting to major trend structures. Terminating early results in underfitting due to insufficient optimization cycles, meaning the parameter matrices have not yet stabilized or captured the subtle, high frequency patterns of the underlying timeline.

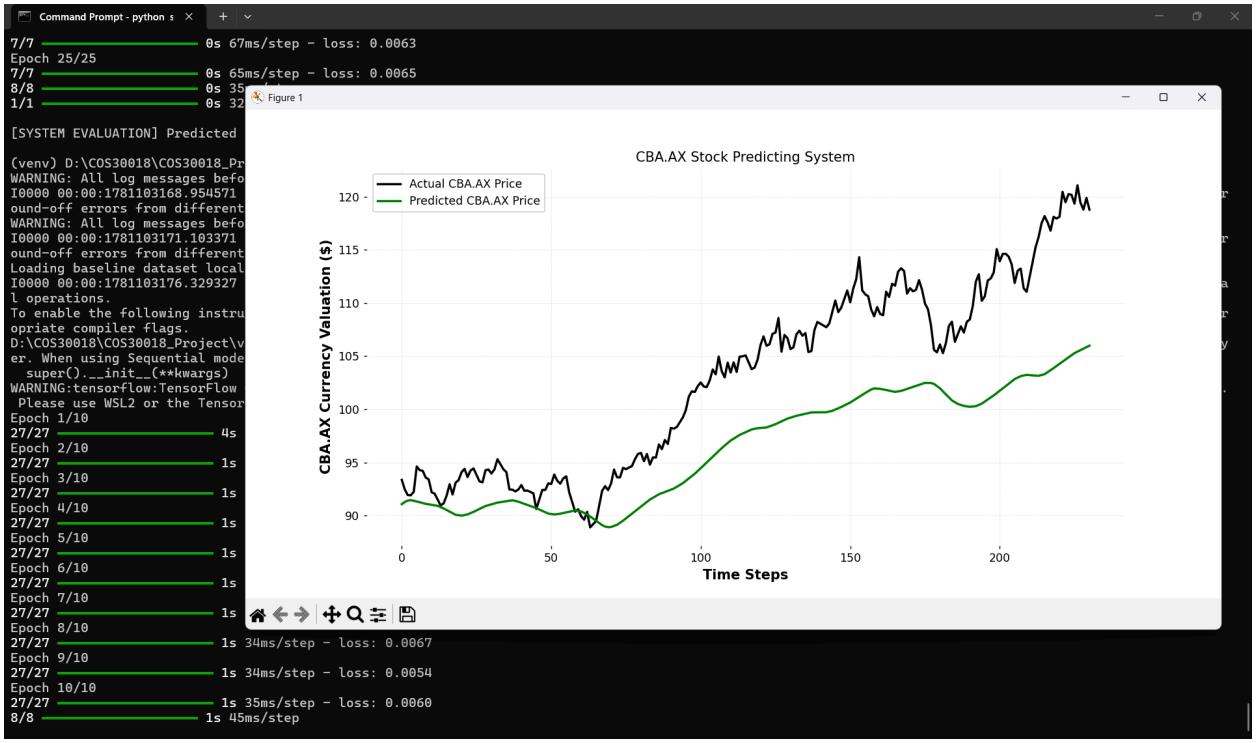

Figure 14 - Experiment 4 output of Run 1 with epochs=10

For run 2, the green line follows the trend quite correctly. It successfully maps the primary directional turning points of CBA.AX while smoothly filtering out transient daily noise.

At 25 epochs, the optimizer has been given enough cycles to minimize macro level tracking errors, yet it has not trained long enough to start memorizing localized statistical noise. This balanced state keeps the network adaptable, ensuring that its mathematical transformations capture generalized market patterns rather than data specific quirks.

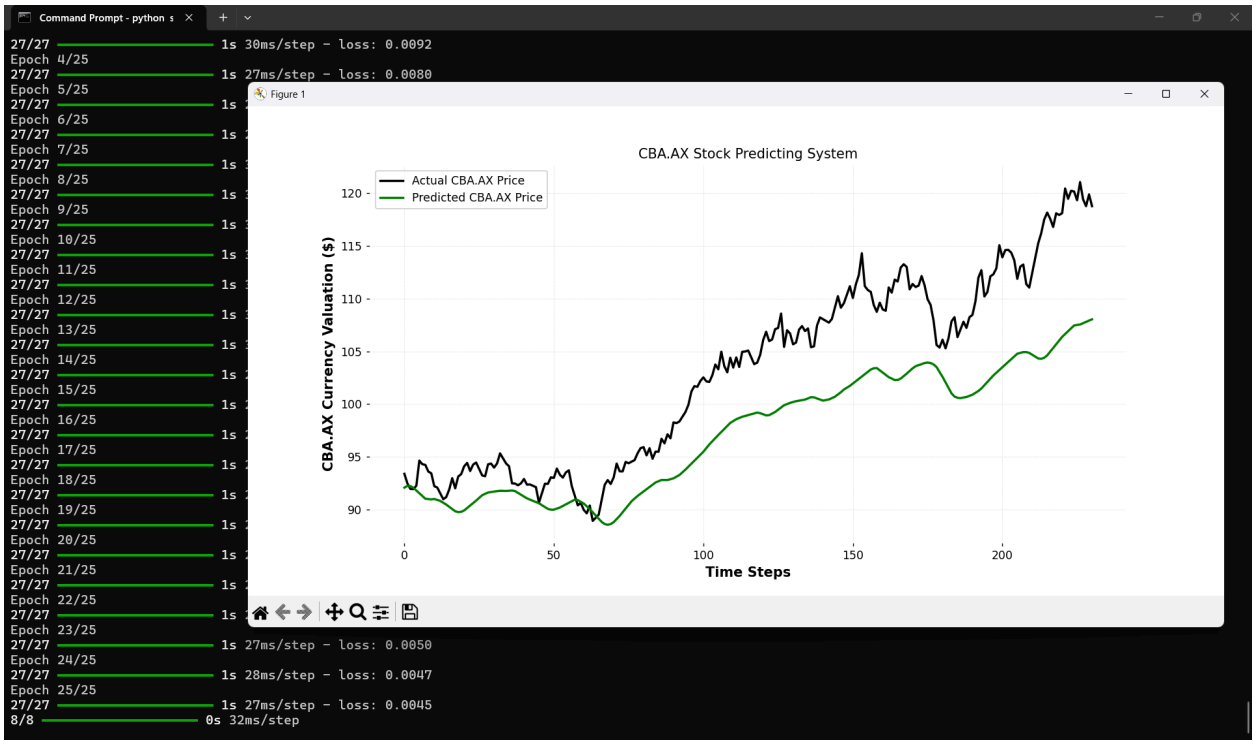

Figure 15 - Experiment 4 output of Run 2 with epochs=25

For run 3, the green line exhibits a highly erratic behavior. The network is matching every minor peak and valley from the past so tightly that the entire curve looks jagged. Even though its prediction looks good, this hyper sensitivity means the model is mistaking temporary daily noise for long term trends, compromising its structural stability.

While an MSE of 0.0021 looks impressive on paper, minimizing training loss across a long horizon poses a major challenge for out-of-sample testing. Allowing a model to continuously process the same training arrays causes the weight layers to fit the specific historical data points too tightly, this is called over-training. The model starts interpreting random market noise as meaningful structural signals, which hurts its ability to handle unseen data.

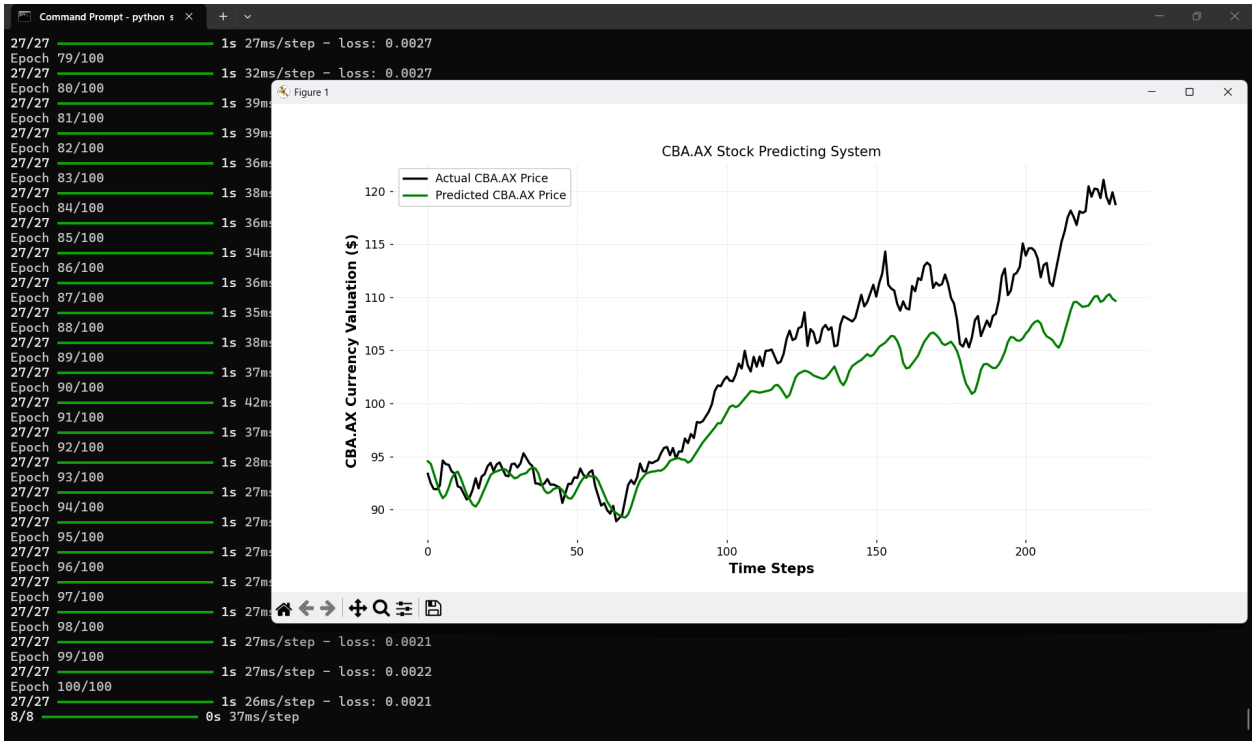

Figure 16 - Experiment 4 output of Run 3 with epochs=100

Appendix: Adam Optimizer

The Adam (Adaptive Moment Estimation) optimizer is an advanced training algorithm designed specifically to update neural network weights efficiently. When dealing with highly volatile sequential data, like financial stock prices, a standard optimizer with a fixed step size often fails. It either jumps around erratically or gets stuck completely.

Adam solves this by computing adaptive learning rates for each individual parameter in the network. Instead of using 1 rigid step size for every weight, Adam dynamically scales its adjustments by tracking 2 key metrics: The directional momentum of the errors (the first moment) and the localized volatility of those errors (the second moment).

Adam provides 3 crucial advantages for our recurrent forecasting network:

- In stock market data, some features variables change drastically while others move slow and steady. Adam automatically shrinks the step size for parameters dealing with violent, noisy spikes while magnifying the step size for parameters tracking quiet, long term trends
- Because Adam retains directional velocity through its momentum term, it possesses the necessary physical inertia to glide right through flat zones and shallow, fake valleys where a standard optimizer's gradient drops to zero and freezes
- Due to these internal self-correcting mechanisms, Adam minimizes the need for tedious manual tuning, ensuring smooth parameter convergence curves even when we aggressively adjust mini-batch dimensions or layer depths.